Ramified Dynamic Operator Reflection

A Proposed Contender 14 for BusyLean

Research note prepared for the BusyLean project

12 October 2026

Base repository commit `1ee50c7d97bbed57a60709f661453c3e213f4db3`

Abstract

We propose a closed computable Lean natural number `contender_14` that strictly exceeds the current BusyLean champion, `contender_13`, without using the predecessor constant in the new definition. The construction is a ramified tower of operator-level reflection over the current dynamic-kind System F_ω normalizer. Its base step upgrades the unary oracle of Contender 13 to a packed countable family and explicitly dominates every visible family coordinate. A strict meta-step then feeds the transformer's whole output family into a full operator-stage closure along a schedule stacked above Contender 12's autonomous schedule. Repeating this meta-step $n+1$ times at input n yields a strict tower of progressively stronger transformers. The seed family exposes all finite additional iterations of the Contender 13 construction.

The proposed Lean development proves, without evaluating any contender,

$$\texttt{contender_13} < \texttt{contender_14}.$$

It also proves the much sharper certificate

$$F_{13}^{42}(C_{13}) < F_{14}(42) < C_{14},$$

where the first exponent denotes forty-two right-associated functional iterations. Moreover, every accepted dynamic-kind program of depth at most n, run with the packed iterated seed family as oracle, has value below $F_{14}(n)$. All numerical definitions are structural and executable in principle. The archive contains Lean source, integration scripts, an axiom and dependency audit extension, and this article. The PDF was compiled and inspected. The preparation environment did not contain Lean 4.34.0, so no kernel-build receipt is fabricated; final acceptance requires running the included commands in the pinned BusyLean checkout.

Contents

1 Executive statement **3**

2 Repository context and admissibility constraints **3**
2.1 Design requirements used here 4

3 Why obvious alternatives are inadequate **4**
3.1 Arithmetic post-composition 4
3.2 Unbounded Busy Beaver maxima 4
3.3 A new calculus without a normalization proof 5

4 Preliminaries: families, transformers, and progressivity **5**

5 Family-aware dynamic-kind reflection **5**

6 A schedule above Contender 12's schedule **6**

7 The strict meta-step **7**

8 The ramified operator tower **7**

9 The iterated Contender 13 seed **8**

10 Definition of F_{14} and the final number **8**

11 Formal comparison theorem **8**

12 A certified large margin **9**

13 Program domination **10**

14 Constructivity and computability **10**

15 Why the growth jump is substantial **11**
15.1 Four independent amplification axes 11
15.2 Schematic ordinal reading 11

16 Axiom and dependency discipline **11**
16.1 Axiom reports 11
16.2 No-laziness dependency closure 12
16.3 Static preflight 12

17 Expected build behavior **12**

18 Integration plan **12**

19 Risk assessment **13**

20 Alternative constructions considered **13**
20.1 One operator stage over $\mathsf{F}_{\mathrm{dyn}}$ 13
20.2 Fixed repeated reflection 14
20.3 A dynamic-kind universe ladder 14
20.4 Large elimination and tree-returning programs 14

21 Further research questions **14**

22 Conclusion **15**

A Lean declaration inventory **15**

B Reproduction checklist **16**

C Artifact integrity **16**

1 Executive statement

The proposed number is

```
def contender_14 : Nat := F14 (F14 42)
```

with the following constructional ingredients:

```
def family13 (n : Nat) : Nat := F13 (F13 n)

def iterateF13 : Nat -> Nat -> Nat
  | 0, n     => n
  | k + 1, n => F13 (iterateF13 k n)

def seed13 : W.Family :=
  fun i n => iterateF13 i (family13 n)

def operatorTower : Nat -> Transformer
  | 0     => fdFamilyTransformer
  | k + 1 => towerStep (operatorTower k)

def F14 (n : Nat) : Nat :=
  operatorTower (n + 1) seed13 n n
```

The base transformer is a family-aware version of the repository's `Fd.reflectFd`. The schedule `tree14` is obtained by placing another 42-level autonomous hyper-closure above `tree12`. The successor operation `towerStep` does not merely compose the previous transformer with a fast arithmetic function; it reruns the repository's full operator-level reflection machinery with the previous transformer's output as the new oracle family.

The main theorem is represented by the Lean declaration

```
theorem contender_13_lt_contender_14 :
    contender_13 < contender_14
```

and follows from three uniform facts:

(i) every level of `operatorTower` is progressive;

(ii) the seed's coordinate zero is `family13`; and

(iii) `F14` is strictly input-increasing.

No numeral is evaluated in this proof.

2 Repository context and admissibility constraints

BusyLean is a Lean 4 continuation of the "name the biggest number" contest. Its operative rules require a closed computable `Nat`, a formal strict comparison with the previous contender, a small permitted axiom set, and a source-level prohibition on defining a new entry from the predecessor constant itself. The repository currently identifies Contender 13 as the champion. That contender reflects all bounded programs of dynamic-kind System F_ω over the function `family12`; it is therefore already far beyond an ordinary tower, Ackermann function, or fixed fast-growing hierarchy at a hand-written ordinal notation.

The relevant predecessor definitions are schematically

$$\begin{aligned} \mathsf{family}_{12}(n) &= F_{12}(F_{12}(n)), \\ F_{13}(n) &= \mathsf{reflectFd}(\mathsf{family}_{12}, n), \\ C_{13} &= F_{13}(F_{13}(42)). \end{aligned}$$

The reflector contains a finite enumeration of terms, a sound bounded type checker, a total normalizer justified by strong normalization, and a maximum of the decoded numeral values. Thus any serious successor should operate on the *construction* of F_{13}, not append an elementary wrapper to C_{13}.

2.1 Design requirements used here

The proposal was designed against the following stronger checklist.

Requirement	Realization
Closed and constructive	Every list, maximum, normalization, tree recursion, and numerical iteration is finite or structurally recursive.
No predecessor constant	`Contender14.lean` uses `F13` and `family12`'s construction, but not `contender_13`.
Formal margin	The proof gives both $C_{13} < C_{14}$ and a 42-extra-iteration lower bound.
Qualitative amplification	Reflection is upgraded from a unary oracle to a family oracle, lifted through operator reflection, and nested by an input-indexed tower of strict meta-steps.
Auditability	Definitions and comparison proofs are separated; the archive adds axiom checks and a transitive dependency-closure check.
Budget plausibility	Earlier large modules are imported precompiled. New definitions are short transparent recursors; proofs reuse generic progressivity theorems.
No informal oracle	The packed family is an ordinary computable function, and program evaluation is the existing total $\mathsf{F}_{\mathrm{dyn}}$ evaluator.

3 Why obvious alternatives are inadequate

3.1 Arithmetic post-composition

A definition such as $2^{C_{13}}$, a tetration of height C_{13}, or even an Ackermann diagonal applied to C_{13} would certainly be larger if it were allowed. It would nevertheless violate the contest's no-laziness principle: the new entry would obtain its strength from the already named constant rather than from a stronger constructive mechanism. Replacing the constant by the definitional equal term $F_{13}(F_{13}(42))$ would evade only a textual check, not the mathematical objection.

3.2 Unbounded Busy Beaver maxima

A true maximum over all terminating programs below a syntax bound is not computable because termination is undecidable. The repository avoids this by using total typed languages and formal normalization theorems. The present proposal keeps that discipline: it reuses the total $\mathsf{F}_{\mathrm{dyn}}$ evaluator and only maximizes over a finite syntax list.

3.3 A new calculus without a normalization proof

A hierarchy of dynamic universes $\texttt{dyn}_0, \texttt{dyn}_1, \ldots$ is a promising qualitative successor and is discussed in the repository's roadmap. However, introducing it responsibly requires extending syntax, kinding, conversion, type inference, semantic interpretation, and the reducibility argument. A large numeral whose central language theorem is merely asserted would not be a valid contest entry. The current proposal instead extracts a substantial new reflection level from machinery that is already normalized and audited, while leaving the dynamic-universe ladder as the leading next base- language project.

4 Preliminaries: families, transformers, and progressivity

A family is a two-argument natural-number function

$$\mathsf{Family} := \mathbb{N} \to \mathbb{N} \to \mathbb{N}.$$

The first argument is an oracle-family coordinate and the second is the numerical input. A transformer maps families to families:

$$\mathsf{Transformer} := \mathsf{Family} \to \mathsf{Family}.$$

The repository's key abstract invariant is progressivity.

Definition 4.1 (Progressive transformer)**.** A transformer T is progressive when

$$\mathsf{Progressive}(T) :\Longleftrightarrow \forall S, i, o, n\, (i \le n \Rightarrow S(i,n) < T(S)(o,n)).$$

The output coordinate o is unrestricted. Only the seed coordinate i must be visible at the current budget.

This deliberately avoids monotonicity in the oracle. Oracle programs may inspect values and branch, so replacing an oracle pointwise by a larger one need not increase a program's output. Progressivity is instead proved from an *explicit occurrence*: a visible seed value is inserted into a finite maximum, or a child value is directly exposed to the parent transformer.

The existing `operatorStage` construction is generic in a progressive base transformer. It first iterates a transformer along the universal well-founded tree `index`; it then recursively splices child transformers and lifts the splice again. The repository proves

$$\mathsf{Progressive}(T) \quad \Longrightarrow \quad \mathsf{Progressive}(\mathsf{operatorStage}(T, a)) \tag{1}$$

for every schedule tree a.

5 Family-aware dynamic-kind reflection

Contender 13's `Fd.reflectFd` accepts one unary oracle $f : \mathbb{N} \to \mathbb{N}$. For operator reflection, a countable family is more natural: child stages must remain separately addressable. The repository already provides a computable pairing code `packFamily` satisfying the intended decode equation for family coordinates and arguments. We therefore define

$$R_{\mathrm{fam}}(S,n) := n + 1 + \max\Big(\max_{i \le n} S(i,n), \max_{t \in \mathsf{ctmsUpTo}(n)} \mathsf{fvalue}(\mathsf{packFamily}(S), n, t)\Big). \tag{2}$$

In Lean this is `reflectFdFamily`:

```
def reflectFdFamily (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum
      (Fd.fvalue (packFamily fs) n) (Fd.ctmsUpTo n))
```

Lemma 5.1 (Visible-coordinate domination)**.** *For every family S and $i \le n$,*

$$S(i,n) < R_{\mathrm{fam}}(S,n).$$

Proof. The prefix maximum contains $S(i,n)$. Hence $S(i,n)$ is at most the left component of the outer maximum. The summand $n+1$ makes the final inequality strict. □

Lemma 5.2 (Program domination)**.** *If $t \in \mathsf{ctmsUpTo}(n)$, then*

$$\mathsf{fvalue}(\mathsf{packFamily}(S), n, t) < R_{\mathrm{fam}}(S,n).$$

Proof. The value is at most the finite maximum over the enumerated terms, which is at most the outer maximum. Again $n+1$ supplies strictness. □

The associated transformer ignores its output coordinate:

$$T_0(S)(o,n) := R_{\mathrm{fam}}(S,n).$$

Visible-coordinate domination is exactly $\mathsf{Progressive}(T_0)$. Thus `fdFamilyTransformer` can be fed to all generic operator-reflection lemmas without changing the existing $\mathsf{F}_{\mathrm{dyn}}$ normalizer.

6 A schedule above Contender 12's schedule

Contender 12 uses

$$\mathsf{tree}_{12} = \mathsf{realize}\big(\mathsf{hyper}(42, \mathsf{encode}(\mathsf{index}))\big).$$

The autonomous notation language contains countably branching trees, syntactic addition and multiplication, ω-powers, derivatives, a finite critical hierarchy, a common-critical-point closure, and higher derivatives. It is intentionally used as tree syntax; no quotient-level ordinal identity is needed for correctness.

The new schedule stacks the same closure once more:

$$\mathsf{tree}_{14} = \mathsf{realize}\big(\mathsf{hyper}(42, \mathsf{encode}(\mathsf{tree}_{12}))\big).$$

The definition is transparent and structurally recursive. Its role is not in the elementary strict comparison—progressivity holds for every schedule—but in making each meta-step a much richer closure than one fixed lift.

Remark 6.1. The exact proof-theoretic ordinal denoted by these raw trees is deliberately not asserted. The construction is valid because `operatorStage` recurses on the actual well-founded Lean tree, not because an informal ordinal label has been proved correct.

7 The strict meta-step

For a progressive transformer T, define

$$\mathsf{towerStep}(T)(S) := \mathsf{operatorStage}(T, \mathsf{tree}_{14})(T(S)). \tag{3}$$

This is `towerStep`. It has two conceptually distinct parts:

(a) first expose the entire output family $T(S)$;

(b) then use that family as the seed of the operator-stage closure of the same transformer T.

The feedback is what gives a clean strict chain.

Theorem 7.1 (Strict domination by one meta-step)**.** *If* $\mathsf{Progressive}(T)$*,* $i \le n$*, and* S *is any family, then*

$$T(S)(i,n) < \mathsf{towerStep}(T)(S)(o,n)$$

for every output coordinate o*.*

Proof. By equation (1), the transformer $\mathsf{operatorStage}(T, \mathsf{tree}_{14})$ is progressive. Apply its progressivity to the seed family $T(S)$ and visible coordinate i. □

Theorem 7.2 (Preservation of progressivity)**.** *If* $\mathsf{Progressive}(T)$*, then* $\mathsf{Progressive}(\mathsf{towerStep}(T))$*.*

Proof. For $i \le n$,

$$S(i,n) < T(S)(i,n) < \mathsf{towerStep}(T)(S)(o,n).$$

The first inequality is progressivity of T; the second is the preceding theorem. □

This proof is important methodologically. It never assumes that reflection is monotone in its oracle. Both strict inequalities are witnessed by the generic progressivity interface.

8 The ramified operator tower

Define a sequence of transformers by

$$T_0 := \mathsf{fdFamilyTransformer},$$
$$T_{k+1} := \mathsf{towerStep}(T_k).$$

This is `operatorTower`. Structural recursion is on k.

Theorem 8.1 (Tower invariant)**.** *For every* $k \in \mathbb{N}$*,* T_k *is progressive.*

Proof. Induct on k. The base case is visible-coordinate domination for `reflectFdFamily`. The successor case is preservation of progressivity by `towerStep`. □

Theorem 8.2 (Strict level chain)**.** *For every* k*, family* S*,* $i \le n$*, and output coordinate* o*,*

$$T_k(S)(i,n) < T_{k+1}(S)(o,n).$$

Proof. This is strict domination by one meta-step, using the tower invariant at level k. □

The word "tower" here is therefore not rhetorical. The Lean development contains a pointwise strict theorem between every two successive levels. Each successor is also substantially more structured than a simple function composition: it invokes a complete operator-stage closure whose base is the entire preceding level.

9 The iterated Contender 13 seed

The predecessor construction is exposed as

$$\mathsf{family}_{13}(n) := F_{13}(F_{13}(n)).$$

Thus $\mathsf{family}_{13}(42) = C_{13}$ definitionally, but the constant `contender_13` is absent from the new executable definition.

Define right-associated iteration by

$$\begin{aligned}\mathsf{iterateF13}(0, x) &:= x,\\ \mathsf{iterateF13}(k+1, x) &:= F_{13}(\mathsf{iterateF13}(k, x)).\end{aligned}$$

The seed family is

$$S_{13}(i, n) := \mathsf{iterateF13}(i, \mathsf{family}_{13}(n)). \tag{4}$$

Coordinate zero is exactly the predecessor family. Coordinate i supplies i further applications of the dynamic-kind reflector. Because visible coordinates satisfy $i \le n$, the input n makes the first $n+1$ such iterations simultaneously available to the packed oracle and to the progressivity maximum.

Lemma 9.1 (Strictness of nonempty F_{13}-iteration)**.** *For all $k, x \in \mathbb{N}$,*

$$x < \mathsf{iterateF13}(k+1, x).$$

Proof. The repository proves $y < F_{13}(y)$ for every y. Induct on k and append this input-growth inequality to the preceding chain. No monotonicity of F_{13} is required. □

10 Definition of F_{14} and the final number

At input n, choose tower level $n+1$, seed it with S_{13}, select output coordinate n, and evaluate at numerical input n:

$$F_{14}(n) := T_{n+1}(S_{13})(n, n). \tag{5}$$

Finally set

$$C_{14} := F_{14}(F_{14}(42)). \tag{6}$$

The outer self-application mirrors the recent BusyLean contenders and is useful because F_{14} is formally input-increasing.

The complete executable spine is only a few transparent definitions. The size comes from the already verified total interpreters and from structural reflection, not from a giant literal term.

11 Formal comparison theorem

Theorem 11.1 (Visible seed bound)**.** *If $i \le n$, then*

$$S_{13}(i, n) < F_{14}(n).$$

Proof. Apply progressivity of T_{n+1} to seed S_{13}, input coordinate i, output coordinate n, and numerical input n. □

At coordinate zero this gives

$$\mathsf{family}_{13}(n) < F_{14}(n). \tag{7}$$

The existing input-growth theorem for F_{13} yields

$$n < F_{13}(n) < F_{13}(F_{13}(n)) = \mathsf{family}_{13}(n).$$

Combining with equation (7) proves

$$n < F_{14}(n). \tag{8}$$

Theorem 11.2 (Main comparison)**.**

$$C_{13} < C_{14}.$$

Proof. At $n = 42$, equation (7) gives

$$C_{13} = \mathsf{family}_{13}(42) < F_{14}(42).$$

Apply equation (8) at input $F_{14}(42)$:

$$F_{14}(42) < F_{14}(F_{14}(42)) = C_{14}.$$

Transitivity completes the proof. □

The corresponding Lean term is intentionally short:

```
theorem contender_13_lt_contender_14 :
    contender_13 < contender_14 :=
  Nat.lt_trans (family13_lt_F14 42)
    (input_lt_F14 (F14 42))
```

The complexity lies in the reusable invariants, not in forcing Lean to normalize either number.

12 A certified large margin

The coordinate $i = 42$ is visible at input 42. Therefore the visible-seed bound gives the exact theorem

$$\mathsf{iterateF13}(42, C_{13}) < F_{14}(42). \tag{9}$$

Since $C_{13} = F_{13}^2(42)$, the left side is $F_{13}^{44}(42)$ under the right-associated iteration convention. Thus the formal chain includes

$$C_{13} < F_{13}^{44}(42) < F_{14}(42) < C_{14}.$$

The archive exposes equation (9) as `fortyTwo_F13_steps_lt_F14`.

This lower bound is useful for two reasons. First, it demonstrates a margin that cannot be dismissed as "plus one." Second, it is independent of any informal ordinal assignment to dynamic-kind System F_ω or to the schedule trees. It follows only from explicit occurrences in finite maxima and from structural progressivity.

13 Program domination

The base family reflector dominates every accepted $\mathsf{F}_{\mathrm{dyn}}$ term in the finite enumeration. The strict tower-level chain places the base below every positive operator-tower level. Consequently:

Theorem 13.1 (Bounded program domination)**.** *If* $t \in \mathsf{ctmsUpTo}(n)$*, then*

$$\mathsf{fvalue}(\mathsf{packFamily}(S_{13}), n, t) < F_{14}(n).$$

Proof. The finite maximum in equation (2) first gives

$$\mathsf{fvalue}(\mathsf{packFamily}(S_{13}), n, t) < T_0(S_{13})(0, n).$$

The strict tower chain, iterated through the positive level $n + 1$, gives

$$T_0(S_{13})(0, n) < T_{n+1}(S_{13})(n, n) = F_{14}(n).$$

□

At the outer budget $F_{14}(42)$ this becomes

$$\begin{aligned} & t \in \mathsf{ctmsUpTo}(F_{14}(42)) \\ \Longrightarrow \quad & \mathsf{fvalue}(\mathsf{packFamily}(S_{13}), F_{14}(42), t) < C_{14}. \end{aligned}$$

This is formalized as `fvalue_lt_contender_14`.

The oracle is more expressive than the unary predecessor oracle. A program can decode an index and query any seed coordinate, while operator-stage nodes replace the seed by families of distinct child outputs. The family interface therefore preserves dynamic access rather than collapsing all child stages to one diagonal function.

14 Constructivity and computability

Proposition 14.1. *`contender_14` denotes a total computable natural number in principle.*

Computational analysis. Each component is total:

(1) `Fd.ctmsUpTo n` is a finite list produced by recursion on n.

(2) The bounded checker and the $\mathsf{F}_{\mathrm{dyn}}$ evaluator are total; accepted terms normalize by the repository's strong-normalization theorem.

(3) `BigNumber.maximum` and `W.prefixMax` inspect finite data.

(4) `runTransformer`, `operatorStage`, `Autonomous.encode`, and `Autonomous.realize` recurse structurally on well-founded inductive trees.

(5) `iterateF13` and `operatorTower` recurse on a natural number.

(6) Function packing is an explicit arithmetic encoding, not an oracle.

No definition is partial, unsafe, noncomputable, opaque, externally implemented, or based on unbounded search. Therefore Lean's evaluator has a finite reduction sequence for the closed term, although its resource demand is astronomical. □

Remark 14.2. "Computable in principle" does not mean feasible to print. BusyLean's own README warns never to evaluate the contenders. Kernel checking verifies the transparent definitions and proof terms without normalizing the final numeral.

15 Why the growth jump is substantial

Only the exact inequalities above are claimed as theorems. Nevertheless, the architecture supports a useful qualitative analysis.

15.1 Four independent amplification axes

A1. Oracle arity. One unary predecessor oracle is replaced by a packed countable family with explicit prefix domination.

A2. Autonomous reflection. A family-aware $\mathsf{F}_{\mathrm{dyn}}$ step is iterated along the W-language's universal index inside each lift.

A3. Operator reflection. Child transformers remain separately selectable and are recursively spliced along `tree14`.

A4. Ramification. The entire operator closure becomes the base of the next level, and the number of levels is $n + 1$ at input n.

The predecessor uses one $\mathsf{F}_{\mathrm{dyn}}$ reflection over `family12`. In contrast, $F_{14}(42)$ uses level 43 of a strict operator tower, with a seed already containing 42 extra F_{13} iterations at visible coordinate 42. The exact lower bound captures only a small, easy-to-state fragment of the full operator-stage computation.

15.2 Schematic ordinal reading

Let $\Lambda_{\mathsf{F}_{\mathrm{dyn}}}$ denote, purely schematically, the closure strength of one family-aware dynamic-kind reflection step, and let α and γ denote the heights represented by `index` and `tree14`. One may view a lift as iterating a $\Lambda_{\mathsf{F}_{\mathrm{dyn}}}$-strength operation along α, while an operator stage recursively lifts spliced operators along γ. Replacing the base transformer by the resulting feedback closure and repeating produces a hierarchy

$$T_0, \quad \mathsf{towerStep}(T_0), \quad \mathsf{towerStep}^2(T_0), \quad \dots$$

whose 43rd level is selected already at input 42.

This paragraph is an explanatory model, not a formal ordinal calibration. A sound calibration would require assigning ordinal semantics to the actual raw-tree operations and proving preservation theorems. The formal Lean comparison deliberately does not depend on such an assignment.

16 Axiom and dependency discipline

The archive extends the repository's audit in two ways.

16.1 Axiom reports

The proposed audit asks Lean to print the axioms of

- `contender_14`;
- `contender_13_lt_contender_14`;
- `fortyTwo_F13_steps_lt_F14`; and

- `fvalue_lt_contender_14`.

No new axiom is introduced by the source. The expected inherited set is the repository-permitted subset of `propext` and `Quot.sound`.

16.2 No-laziness dependency closure

The existing meta-level command `#assert_not_in_closure` traverses all constants reachable through declaration types and bodies. The integration appendix checks that `contender_14` does not depend on any predecessor constant `contender_13`,...,`contender_5`. It is allowed to depend on `F13`, `family12`, the normalizer, and the generic reflection machinery: those are predecessor *constructions*, exactly as the contest rule permits.

16.3 Static preflight

The supplied Python preflight strips nested Lean comments and strings, then checks the executable source for forbidden tokens and predecessor constants. It also verifies the expected declaration set and records SHA-256 hashes. This is useful before integration but is explicitly labeled as not being a kernel check.

17 Expected build behavior

The repository permits earlier contender modules to be precompiled when timing a new entry. Under that convention the definition phase imports `Contender13`, adds one finite maximum, several small transparent recursors, and no new normalization proof. The comparison phase is dominated by generic theorem instantiation and short inductions over natural tower levels. Thus the design is intentionally conservative with respect to the 15-second definition and 60-second proof budgets.

The archive pins the same toolchain as the base repository:

`leanprover/lean4:v4.34.0`.

The intended commands after applying the overlay are

```
python3 scripts/check.py
lake env lean test/Contender14Audit.lean
```

A successful run should be retained as a build receipt. Because the execution environment used to create this archive had no Lean binary and could not fetch the 4.34.0 toolchain, this article does not claim a completed kernel build. That limitation is explicit rather than hidden behind a fabricated log.

18 Integration plan

The archive is an overlay, not a fork. It contains

- `BusyLean/Contender14.lean`;
- `BusyLean/Comparison14.lean`;
- `test/Contender14Audit.lean`;

- an append block for the repository-wide audit;
- scripts to copy the overlay and run checks;
- a unified integration patch and documentation;
- this LaTeX article and compiled PDF.

The root module should import the two new modules after `BusyLean.Comparison13`. The timing script should set

```
PREVIOUS = "BusyLean.Comparison13"
CURRENT_DEF = "BusyLean.Contender14"
```

and the audit should add the declarations listed in the preceding section.

19 Risk assessment

Risk	Consequence	Mitigation
Lean elaboration mismatch	Source needs a local syntactic repair.	The code copies theorem shapes already used by Comparisons 11–13 and minimizes tactic use. Run the pinned checker before acceptance.
Unexpected axiom dependency	Contest audit fails.	No new semantic theorem is introduced; all proofs use existing progressivity and elementary `Nat` lemmas. Explicit `#print axioms` entries are supplied.
Definition budget regression	Entry violates timing rule.	No new object language or normalization proof; previous modules are precompiled by the contest's timing convention.
No-laziness audit failure	Entry is inadmissible.	The executable module does not mention predecessor constants. A transitive closure assertion is supplied.
Overstated proof-theoretic size	Misleading research claim.	Only strict numeric and program bounds are theorem claims. Ordinal discussion is labeled schematic.
Oracle nonmonotonicity	Invalid comparison proof.	All inequalities use explicit progressivity; no oracle monotonicity is assumed.

20 Alternative constructions considered

20.1 One operator stage over $\mathsf{F}_{\mathrm{dyn}}$

The smallest natural successor would instantiate Contender 12's generic operator stage with family-aware $\mathsf{F}_{\mathrm{dyn}}$ reflection and seed it with `family13`. That already gives a large strict improvement.

The present proposal goes further by proving a strict feedback step and iterating that whole step through an input-indexed tower.

20.2 Fixed repeated reflection

One could iterate `Fd.reflectFd` 42 times. This is simple and formally large, but it collapses all child stages to a unary chain and fails to exploit the repository's family and operator infrastructure. It also gives a much weaker structural explanation of the growth margin.

20.3 A dynamic-kind universe ladder

A hierarchy $\texttt{dyn}_j$ would be a stronger base-language advance. It is likely the most important next long-term direction. It is not used here because a valid implementation requires a new normalization model and sound checker, not just syntax. The current operator tower can later be re-based on such a language: replace `fdFamilyTransformer` with the progressive reflector of the verified ladder while keeping the rest of the construction.

20.4 Large elimination and tree-returning programs

A calculus with inductive trees and large elimination could internalize the world/model bootstraps explored by earlier proposals. This may eventually outgrow any amplifier over $\mathsf{F}_{\mathrm{dyn}}$. It is again a base-language research program, not a small safe overlay.

21 Further research questions

1. **Kernel-check and benchmark the overlay.** Resolve any version- specific elaboration issue, record clean definition/proof times, and preserve complete axiom output.
2. **Formalize a generic strict meta-closure theorem.** Abstract `towerStep` away from `tree14` and prove reusable results for any schedule and progressive transformer, including closed forms for finite level chains.
3. **Prove stronger program bounds at every tower level.** Track not merely the base $\mathsf{F}_{\mathrm{dyn}}$ program maximum but programs relative to each derived child family, with explicit embedding lemmas through `operatorStage`.
4. **Develop ordinal semantics for the raw autonomous trees.** Define a quotient or simulation relation, prove monotonicity of `add`, `mul`, `pow`, `derivative`, and `hyper`, and derive certified lower bounds for `tree12` and `tree14`.
5. **Implement the dynamic-universe ladder.** Add $\texttt{dyn}_0, \ldots, \texttt{dyn}_{n-1}$ at syntax budget n, define the level-indexed semantic dependent sums, and extend strong normalization and inference soundness. A diagonal reflector over this language should be the next qualitative base jump.
6. **Universe-polymorphic compression.** Investigate whether the finite ladder can be represented by one level-indexed kind constructor while preserving a decidable, predicative semantic model and fast elaboration.
7. **Tree-capable dynamic calculus.** Combine dynamic kinds with W-types or strictly positive inductives, prove normalization with large elimination where sound, and expose generated trees to the reflection schedule.

8. **Higher meta-types.** Formalize certified progressive transformers as a subtype and iterate the lift at the level (Transformer → Transformer), then at higher finite meta-types. Compare this genuine type-level tower with the present same-type feedback tower.

9. **Minimality and redundancy.** Determine which components of `tree14`, the seed iteration, and the feedback composition are needed for the strongest formal lower bounds under the timing budget.

10. **Machine-checked no-laziness specification.** Replace the current blacklist with a positive provenance policy describing which predecessor families and constructors may occur, and prove the policy stable under refactoring.

11. **Independent normalization replay.** Export the $\mathsf{F}_{\mathrm{dyn}}$ strong- normalization dependency graph and replay its critical semantic lemmas in a second small checker or a minimized Lean environment.

12. **Compare amplifier versus language jumps.** Build a formal embedding framework that can prove when an object-language extension dominates any bounded number of reflection amplifiers over the old language.

22 Conclusion

The proposed Contender 14 is a direct, auditable successor to the current BusyLean architecture. It avoids the prohibited shortcut of applying a fast function to the old constant. Instead it upgrades the old reflector to a family oracle, performs autonomous operator reflection, feeds the result back as the next seed, and repeats the entire strict closure at an input-indexed number of levels. The comparison theorem is elementary once the right invariant—progressivity—is isolated, but the number it certifies is extraordinarily large.

The most informative exact certificate is not merely $C_{13} < C_{14}$ but

$$F_{13}^{44}(42) < F_{14}(42) < C_{14}.$$

The full construction also dominates all bounded $\mathsf{F}_{\mathrm{dyn}}$ programs using the packed iterated seed family. This makes the proposal a substantial construction-level advance while remaining small enough to audit and integrate. The remaining mandatory step is a clean Lean 4.34.0 kernel build in the base repository, using the supplied scripts and audit extensions.

A Lean declaration inventory

Declaration	Purpose
`reflectFdFamily`	Family-aware $\mathsf{F}_{\mathrm{dyn}}$ reflector with a finite prefix maximum.
`fdFamilyTransformer`	Base progressive transformer.
`tree14`	Autonomous schedule stacked above `tree12`.
`towerStep`	Feedback through a full operator-stage closure.
`operatorTower`	Natural-number-indexed strict transformer tower.

Declaration	Purpose
`family13`	Predecessor construction as a function, not a constant.
`iterateF13`	Right-associated finite iteration of `F13`.
`seed13`	Family of further `F13` iterations.
`F14`	Input-indexed tower output.
`contender_14`	Final closed natural number.
`lt_reflectFdFamily`	Generic maximum-to-reflector strict inequality.
`oracle_lt_reflectFdFamily`	Visible family coordinate bound.
`fvalue_lt_reflectFdFamily`	Bounded $\mathsf{F}_{\mathrm{dyn}}$ program bound.
`fdFamilyTransformer_progressive`	Base progressivity.
`transformer_lt_towerStep`	Strict feedback-step theorem.
`towerStep_progressive`	Preservation of progressivity.
`operatorTower_progressive`	Invariant for every tower level.
`operatorTower_level_lt_succ`	Strict pointwise level chain.
`seed13_lt_F14`	Uniform visible-seed lower bound.
`fortyTwo_F13_steps_lt_F14`	Exact contest-input margin.
`contender_13_lt_contender_14`	Required comparison.
`fvalue_lt_contender_14`	Outer-budget program domination.

B Reproduction checklist

1. Check out BusyLean commit `1ee50c7d97bbed57a60709f661453c3e213f4db3`.
2. Confirm `lean-toolchain` contains `leanprover/lean4:v4.34.0`.
3. Run `scripts/apply_overlay.sh /path/to/BusyLean` from this archive.
4. Append the supplied audit block to the repository audit. Its path is `patches/Audit14.append.lean`. Alternatively, apply the unified patch in `patches/integration.patch`.
5. Run `python3 scripts/check.py`.
6. Run `lake env lean test/Contender14Audit.lean`.
7. Save stdout, wall-clock timings, Lean version, operating system, and commit hash as the acceptance receipt.
8. Confirm the dependency-closure assertion excludes all predecessor constants and that the printed axiom sets are permitted.

C Artifact integrity

The archive includes `SHA256SUMS.txt`, generated after PDF compilation and quality inspection. The file `artifacts/static_audit.txt` records the preflight hashes of the two principal Lean modules. The verification-status file distinguishes completed checks from checks that still require a pinned Lean installation.

References

[1] V. Reshetnikov, *BusyLean: name the biggest number, in Lean*, GitHub repository, 2026. Base commit used here: `1ee50c7d97bbed57a60709f661453c3e213f4db3`.

[2] C. Roux, *name-the-biggest-number*, Coq contest repository.

[3] S. Aaronson, *Who Can Name the Bigger Number?*, expository essay.

[4] J.-Y. Girard, *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*, 1972.

[5] W. W. Tait, Intensional interpretations of functionals of finite type I, *Journal of Symbolic Logic* 32 (1967), 198–212.